RESEARCH MEETING

ThunderServe: Heterogeneous GPU-Aware LLM Serving

Phase-Level Deployment in Heterogeneous Clouds

Yu-Jui Liang

Advisor: Hsiang-Yun Cheng

National Taiwan University | 21 Oct 2026

Presentation Roadmap

THUNDERSERVE / OVERVIEW

1. **Background / Motivation**
   GPU capabilities, inference phases and network heterogeneity.
2. **Methodology**
   Deployment search, replica orchestration and runtime adaptation.
3. **Experimental Results**
   SLO attainment, throughput, ablations and model validation.
4. **Discussion / Conclusion**
   Scope, open questions and main findings.

Source: arXiv:2502.09334 (ThunderServe).

Prefill and Decode Have Different Bottlenecks

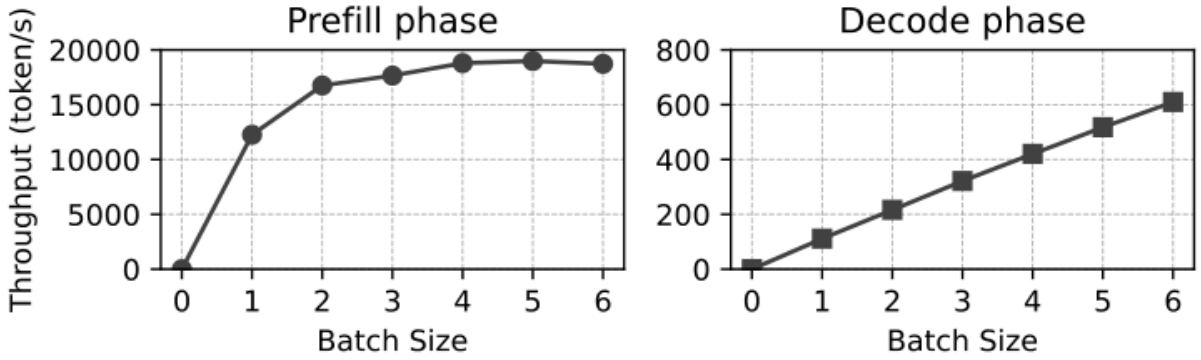


Figure 2. Effects of batching on different phases (LLaMA-7B with each input having a sequence length of 1024).

Fig. 2: LLaMA-7B; 1024 input tokens per request

- **Prefill:** process the prompt, build KV cache and generate the first token.
- In this test, token throughput soon saturates as batch size grows.
- **Decode:** reuse KV cache and generate one new token per step.
- Batching amortizes weight reads and improves decode throughput.

Prefill is usually compute-bound; decode is often memory-bandwidth-bound.

Different GPUs Favor Different Inference Phases

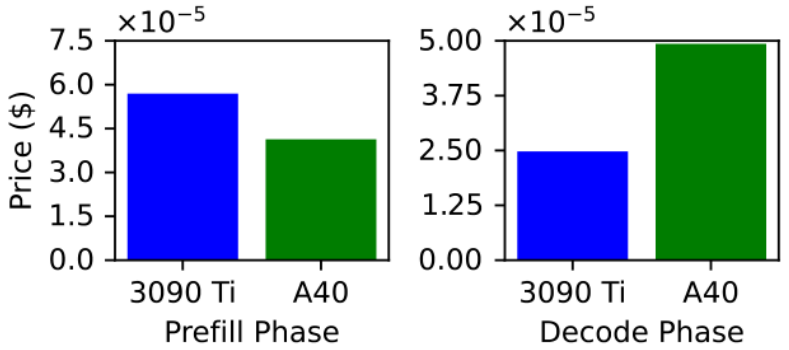


Figure 1. Prefill and decode prices for a single request with input and output lengths of 512 and 16 on 3090Ti and A40.

Fig. 1: Per-request cost; input 512 / output 16 tokens

- A40 has lower prefill cost in this experiment.
- Its compute capability favors prompt processing.
- 3090 Ti has lower decode cost in this experiment.
- Its higher memory bandwidth favors token generation.

Choose GPUs by phase, workload and rental cost, rather than a single GPU ranking.

GPU Specifications in Table 1

GPU	Memory bandwidth	Peak FP16	Memory	Price / GPU
A100	2 TB/s	312 TFLOPS	80 GB	$1.753/hr
A6000	768 GB/s	38.7 TFLOPS	48 GB	$0.483/hr
A5000	626.8 GB/s	27.8 TFLOPS	24 GB	$0.223/hr
A40	696 GB/s	149.7 TFLOPS	48 GB	$0.403/hr
3090Ti	1008 GB/s	71 TFLOPS	24 GB	$0.307/hr

A40's 149.7 TFLOPS is the compute side of the previous cost figure; 3090Ti's 1008 GB/s is the memory-bandwidth side. Eight A100s at $1.753/hr match the later $14.024/hr in-house budget.

Use these per-GPU numbers for capability; Section 5.1's $13.542/hr is the rented 32-GPU bundle, not the sum of this column.

Source: Table 1 and Section 5.1.

Cloud Networking Is Highly Heterogeneous

- Left: 32 cloud GPUs have uneven connectivity across GPU pairs.
- Right: 8 in-house A100 GPUs have much faster links; color scales differ.
- Phase splitting adds a transfer of prompt KV cache between replicas.
- Low-bandwidth paths can erase the benefit of specializing GPUs by phase.

Deployment must account for both GPU capability and the communication topology.

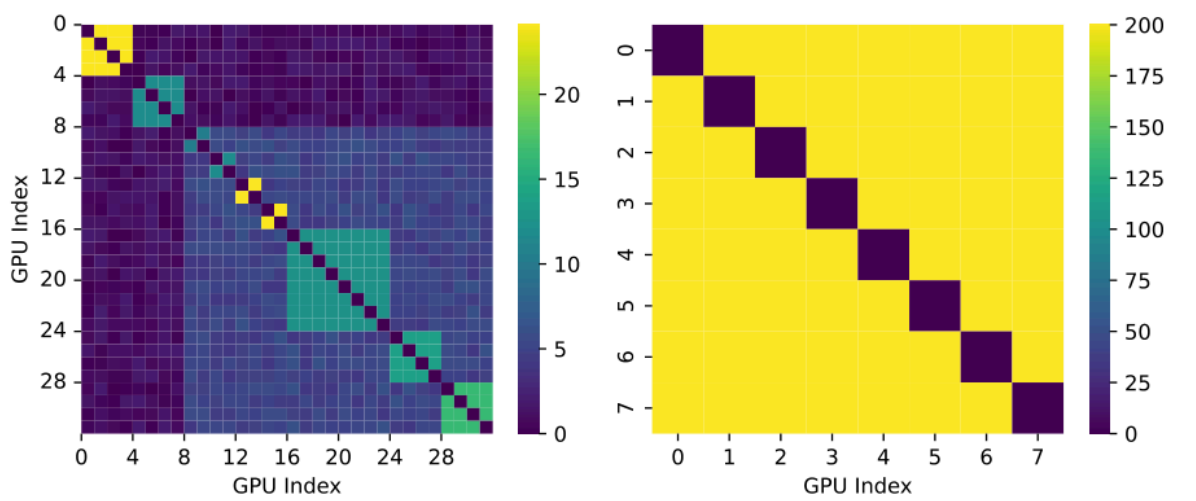


Figure 13. Heat map of inter-connection bandwidth matrix in the cloud (left) and in-house (right) settings.

Fig. 13: Pairwise bandwidth measured with NCCL

Three Challenges for Phase Splitting in Clouds

THUNDERSERVE / BACKGROUND & MOTIVATION

1. **Limited, heterogeneous GPU inventory**
   Group available GPUs into replicas and assign each replica a phase.
2. **Slow, heterogeneous network links**
   Account for TP / PP communication and the extra KV cache transfer.
3. **Changing workloads and GPU availability**
   Adjust resource allocation without repeatedly reloading model weights.

ThunderServe jointly plans deployment and routing, then adapts the running service.

Source: Introduction, challenges 1–3.

ThunderServe: System Overview

THUNDERSERVE / METHODOLOGY

- The scheduler inputs model configuration, workload patterns, GPU types and bandwidth matrix.
- The workload profiler monitors average prompt and output lengths.
- The task coordinator dispatches requests across prefill and decode replicas.
- Workload shifts or GPU heartbeat timeouts trigger lightweight re-scheduling.

The deployment plan links scheduling decisions to online request dispatching.

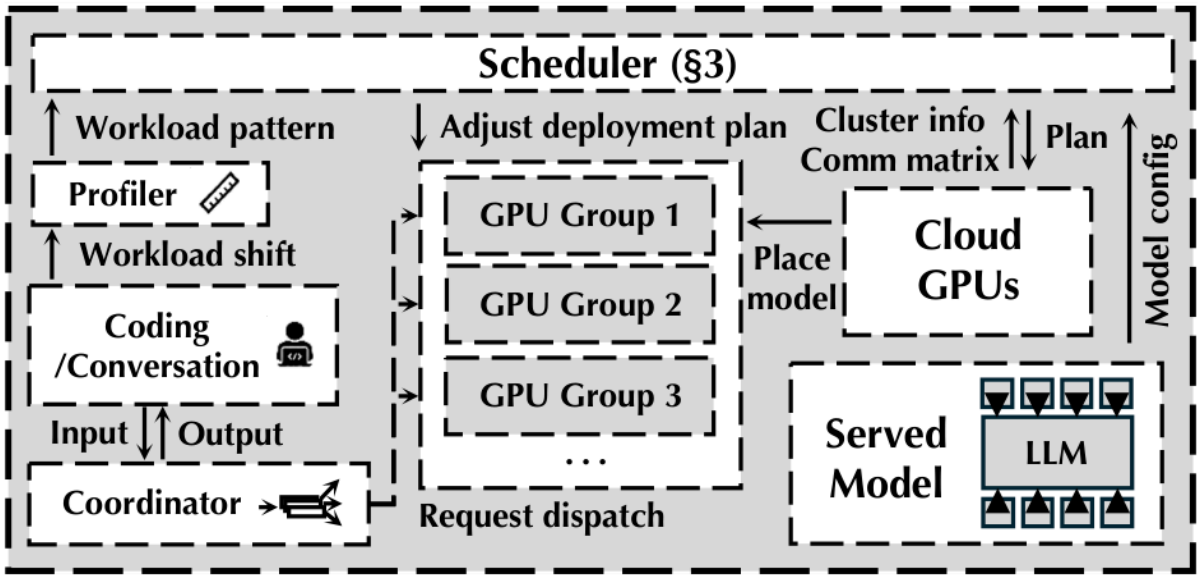


Figure 15. System overview of ThunderServe.

Fig. 15: Scheduler, workload profiler, task coordinator

A Deployment Plan Has Four Components

THUNDERSERVE / METHODOLOGY

1 / GROUP CONSTRUCTION

Model serving groups

Partition available GPUs into groups; each group serves one model replica.

2 / PHASE DESIGNATION

Prefill or decode

Specify which inference phase each model serving group serves.

3 / PARALLEL CONFIGURATION

Parallelism strategies

Determine tensor and pipeline model parallelism for each replica.

4 / ORCHESTRATION

Request routing

Determine how requests are routed through prefill and decode replicas.

Upper-level: group construction + phase designation; lower-level: parallel configuration + orchestration.

Source: ThunderServe, Section 3.1 and Figure 3.

Scheduling as Two-Level Hierarchical Optimization

THUNDERSERVE / METHODOLOGY

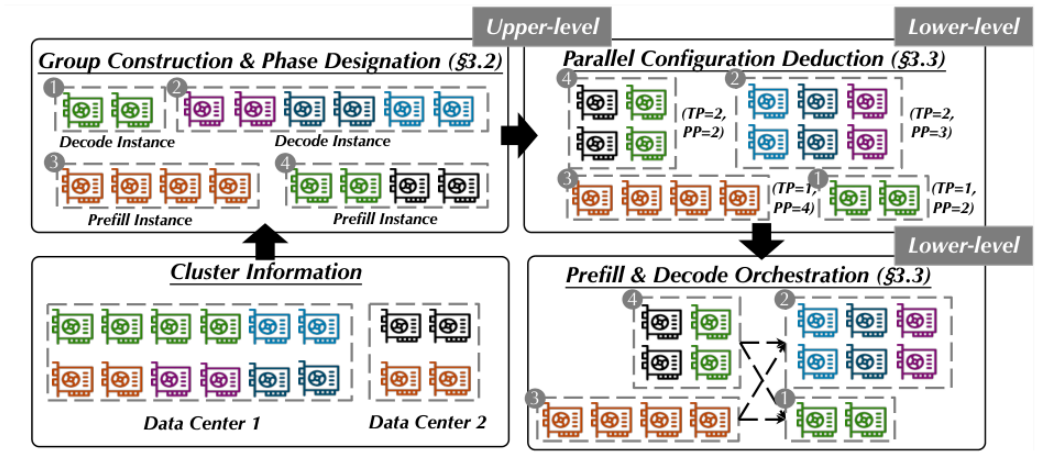


Figure 3. Workflow of our scheduling algorithm.

Fig. 3: Workflow of the scheduling algorithm

- Upper-level optimizes group construction and phase designation.
- Lower-level optimizes parallel configuration and replica orchestration.
- Given a candidate upper-level solution, the lower-level returns its SLO attainment.
- The upper-level search uses that performance feedback to improve the deployment plan.

Objective: maximize the fraction of requests meeting the end-to-end latency deadline.

Initialization and Feasibility in Tabu Search

THUNDERSERVE / METHODOLOGY

INITIAL SOLUTION

Hierarchical Clustering

Cluster GPUs based on their inter-connection bandwidth matrix; treat each cluster as a model serving group.

EARLY CHECK

GPU memory limit

Eliminate a neighbor if any serving group cannot store one copy of the model parameters.

PHASE DESIGNATION

Random initialization

Randomly assign the initial prefill / decode phase of each group.

SEARCH COMPLEXITY

Combinatorial search

The paper motivates tabu search via a JSSP analogy (Appendix A); the search is heuristic.

Bandwidth-aware initialization and memory pruning reduce wasted search.

Source: Section 3.2, Algorithm 1, Appendix A.

Four Neighbor Operations in Tabu Search

THUNDERSERVE / METHODOLOGY

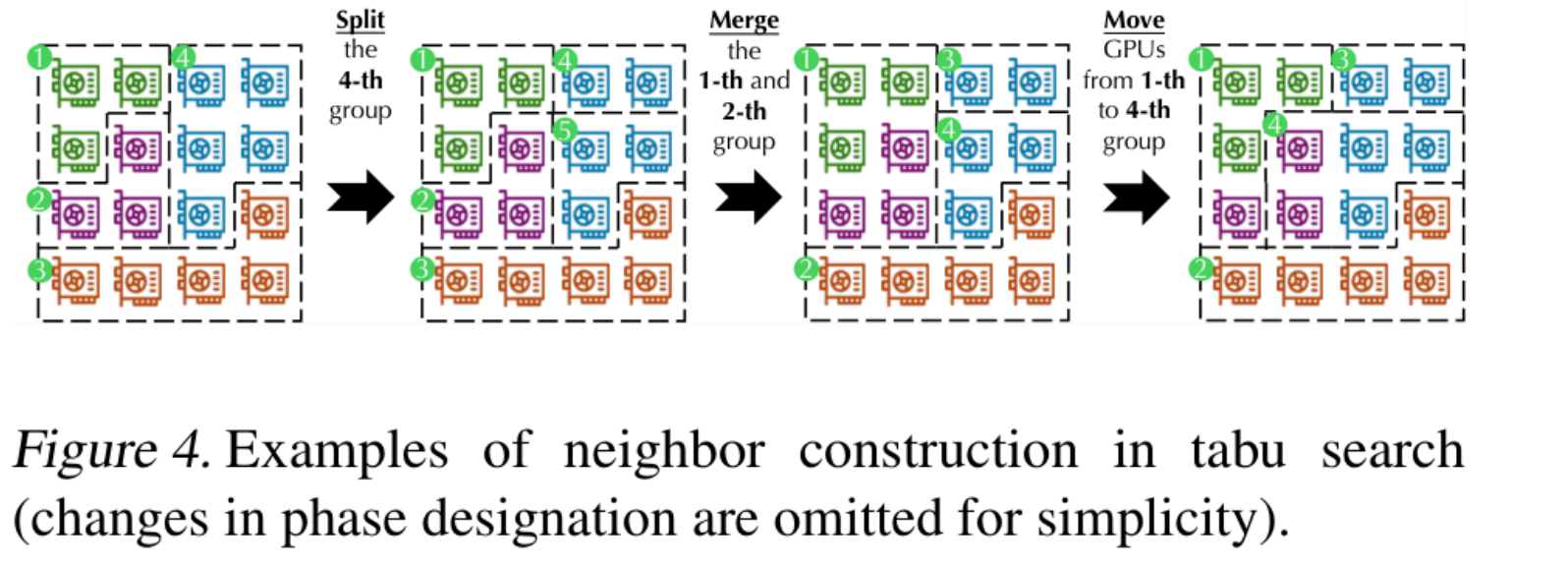


Figure 4. Examples of neighbor construction in tabu search (changes in phase designation are omitted for simplicity).

Fig. 4: Neighbor construction in tabu search

- **Flip:** change a serving group's prefill or decode phase.
- **Split:** divide one GPU group into two.
- **Merge:** combine two GPU groups.
- **Move:** transfer GPUs of a given type between groups.

Tabu search explores changes in both GPU grouping and phase designation.

Neighbor Updates in Notation

THUNDERSERVE / METHODOLOGY

$g_{i,t}$: number of type-t GPUs in group i, with types $t \in \{1, \ldots, T\}$. Flip only changes a group's phase and has no g update.

Split group s by a random ratio r:

$$g_{s'_1,t} \leftarrow \lfloor g_{s,t} \times r \rfloor, \qquad g_{s'_2,t} \leftarrow g_{s,t} - g_{s'_1,t}.$$

Merge groups i and j: $g'_{\text{merged},t} \leftarrow g_{i,t} + g_{j,t}$.

Move m_t GPUs of type t from i to j:

$$g'_{i,t} \leftarrow g_{i,t} - m_t, \qquad g'_{j,t} \leftarrow g_{j,t} + m_t.$$

New groups from split or merge receive a random phase. A neighbor is dropped if some group cannot store one model copy.

These three updates are the grouping half of the four neighbor operations.

Source: Section 3.2. The formulas are unnumbered in the paper.

Tabu Search: Evaluation and Update

THUNDERSERVE / METHODOLOGY

Algorithm 1: Upper-level tabu search

1: **function** TabuSearch($N_{\text{step}} = 100$, $N_{\text{nghb}} = 10$, $N_{\text{mem}} = 5$):
2: Initialize the current solution x
3: Initialize tabu list $T \leftarrow [\,]$, best solution $x_{\text{best}} \leftarrow x$
4: */* Iterative Neighborhood Search */*
5: **for** N_{step} search steps **do**
6: Construct N_{nghb} neighbors of x and exclude those in T to form the neighborhood set $\mathcal{N}$ for navigation
7: $x' \leftarrow \arg\max_{x'' \in \mathcal{N}} f(x'')$
8: **if** $f(x') > f(x_{\text{best}})$ **then** $x_{\text{best}} \leftarrow x'$
9: $T.\text{append}(x')$
10: **if** $\text{len}(T) > N_{\text{mem}}$ **then** $T \leftarrow T[-N_{\text{mem}}:]$
11: $x \leftarrow x'$
12: **end for**
13: **return** x_{best}
14: **end function**

x: GPU grouping + phase roles; $f(x)$: SLO attainment from the lower-level problem; T: tabu list.

Source: Algorithm 1, Section 3.2, PDF p. 5. Transcribed with original line numbers; notation typeset for slides.

Deducing Parallel Configurations for Each Group

THUNDERSERVE / METHODOLOGY

TENSOR MODEL PARALLELISM

TP: split work within layers

Disallow cross-node TP; Appendix B additionally limits TP to GPUs of a single type.

PIPELINE ROUTING

Dynamic programming

Identify pipeline communication paths that minimize cross-stage communication cost.

PIPELINE MODEL PARALLELISM

PP: split layers into stages

Account for different GPU memory capacities and computing abilities.

PARALLEL CONFIGURATION

Phase-specific objective

Select the lowest predicted latency for prefill; highest predicted throughput for decode among candidates.

Deduce each group's parallel configuration before prefill / decode orchestration.

Source: Section 3.3 and Appendix B; cost model from HexGen.

Algorithm 2: Parallel Configuration Search (1/2)

THUNDERSERVE / METHODOLOGY

1: **Initialize:** group formation $G = \{G_1, G_2, \ldots, G_g\}$, minimum number of single-type GPUs in the group $T = \{T_1, T_2, \ldots, T_g\}$, cluster information I, model configuration M
2: $\text{model_parallel_configurations} \leftarrow [\,]$
3: **for** i in $\text{len}(G)$ **do**
4: $\quad \text{plan_list} \leftarrow [\,]$
5: $\quad$ */* Limit TP within Single-type GPUs */*
6: $\quad$ **for** $TP \in \{1, 2, \ldots, T_i\}$ **do**
7: $\quad\quad$ **for** $PP \in \{1, 2, \ldots, G_i.\text{num_gpus}/T_i\}$ **do**
8: $\quad\quad\quad$ */* Route Pipeline Communication */*
9: $\quad\quad\quad \text{plan} \leftarrow \text{Dynamic_Programming}(I, TP, PP)$
10: $\quad\quad\quad$ */* Generate Pipeline Partition */*
11: $\quad\quad\quad \text{plan} \leftarrow \text{Pipeline_Partition}(M, \text{plan})$
12: $\quad\quad\quad$ **if** $G_i.\text{type}$ is prefill **then**
13: $\quad\quad\quad\quad C \leftarrow \text{latency}(\text{plan})$
14: $\quad\quad\quad$ **else**
15: $\quad\quad\quad\quad C \leftarrow \text{throughput}(\text{plan})$
16: $\quad\quad\quad$ **end if**
17: $\quad\quad\quad \text{plan_list.append}((C, \text{plan}))$

Source: Appendix B, Algorithm 2, PDF p. 13; original lines 1–17. All 29 lines retained across two slides; indentation clarified.

Algorithm 2: Parallel Configuration Search (2/2)

18:	**end for**
19:	**end for**
20:	**if** G_i. type is prefill **then**
21:	*/* Select Latency Optimal Plan */*
22:	plan $\leftarrow \min(C)$ in plan_list
23:	**else**
24:	*/* Select Throughput Optimal Plan */*
25:	plan $\leftarrow \max(C)$ in plan_list
26:	**end if**
27:	model_parallel_configurations. append(plan)
28:	**end for**
29:	**return** model_parallel_configurations

Lines 18–19 finish PP / TP enumeration; lines 20–27 choose and store one plan for this group.

Lines 22 / 25 select the plan associated with the smallest / largest C. Line 28 ends the group loop; line 29 returns all selected configurations.

Source: Appendix B, Algorithm 2, PDF p. 13; original lines 18–29. All 29 lines retained across two slides; indentation clarified.

Orchestrating Prefill and Decode Replicas

THUNDERSERVE / METHODOLOGY

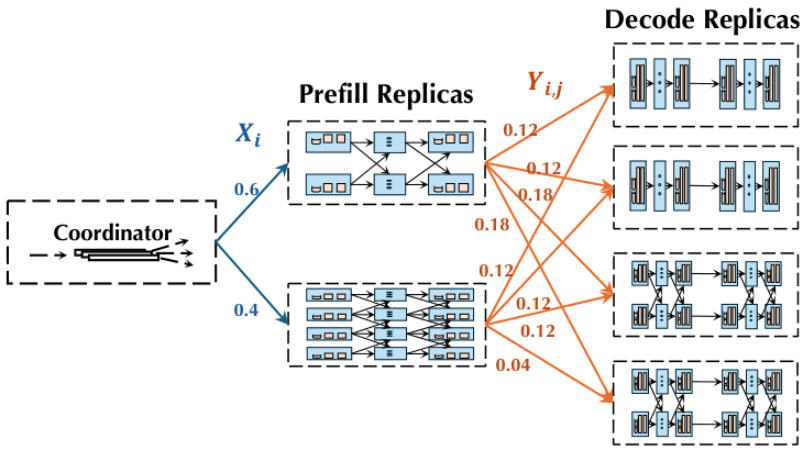


Figure 5. An example orchestration of prefill and decode replicas.

Fig. 5: Example prefill-to-decode orchestration

- A request executes on a prefill replica and then a decode replica.
- KV cache must be transmitted between the two replicas.
- DistServe's inference task simulator estimates pairwise SLO attainment.
- The resulting matrix D_{ij} scores each prefill–decode pair, including KV transfer cost.

Request routing must account for heterogeneous inter-replica communication.

Modeling KV Cache Communication Cost

THUNDERSERVE / METHODOLOGY

Paper Eq. (1): alpha-beta communication model

$$T_{ij}^{(\text{kv comm})} = \alpha_{ij} + \frac{2bshN_{\text{bytes}}}{\beta_{ij}}$$

b: batch size; s: sequence length; h: block hidden size; N_{bytes}: bytes per KV element.
α_{ij}: startup latency; β_{ij}: bandwidth in bytes/s; factor 2: keys and values.

- Transfer time grows with payload size and falls with link bandwidth.
- **Scope note:** the printed equation has no layer-count term.
- For full-model KV cache, tensor accounting gives $V_{\text{KV}} = 2Lbsh_{\text{KV}}q$ bytes before metadata (L: layers; h_{KV}: total KV-head width; q: bytes/element).

Account for transferred layers, KV heads and sharding when applying the model.

Source: Section 3.3, Eq. (1). Full-model tensor accounting is an explanatory clarification.

Replica Routing Objective and Formulation Caveats

THUNDERSERVE / METHODOLOGY

D_{ij}: simulated SLO attainment for prefill replica i and decode replica j.

$$\max_{X,Y} \sum_{i=1}^{m} \sum_{j=1}^{n} X_i Y_{ij} D_{ij}$$

Printed constraints: $\sum_{i=1}^{m} X_i = 1$, $\sum_{j=1}^{n} Y_{ij}$ for all i, $X_i \geq 0$, $Y_{ij} \geq 0$.

- X_i: fraction of incoming requests assigned to prefill i. Y_{ij}: fraction routed to decode j, conditional on prefill i.
- The paper calls this a two-stage transportation problem solved by LP.
- **The print is incomplete:** $X_i Y_{ij}$ is bilinear, the Y sum has no equality, and capacity constraints are absent.

The routing idea is clear; the printed equation alone is insufficient to reproduce the LP.

Source: Section 3.3. Caveats are observations on the supplied PDF, p. 6.

Network-Aware Deployment Plans

THUNDERSERVE / METHODOLOGY

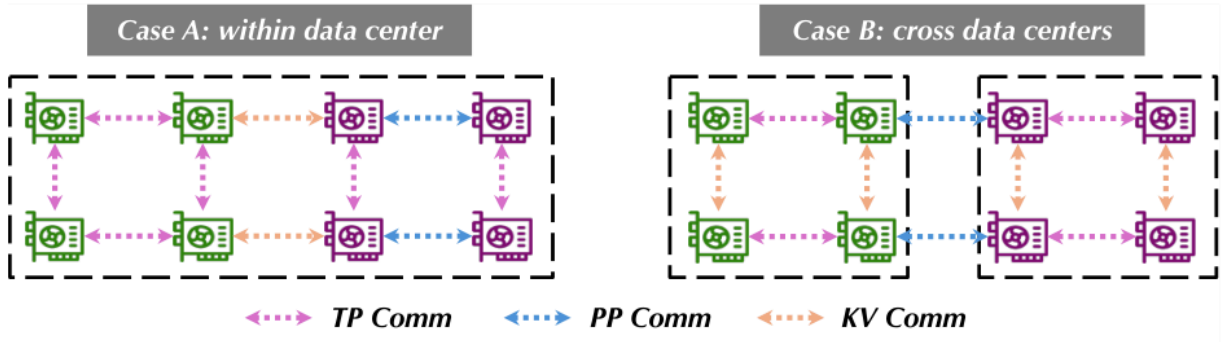


Figure 17. ThunderServe deployment plans on different cases.

Fig. 17; Appendix H: LLaMA-30B, 4 A40 + 4 3090 Ti

- At 40 Gbps, use 4-A40 for prefill and 4-3090Ti for decode.
- Throughput: 3292 vs. 1610 tokens/s for the non-disaggregating baseline.
- At 5 Gbps, both phases use 2-A40 and 2-3090Ti across instances.
- 2196 tokens/s ($1.4\times$ baseline): KV stays within instances; PP crosses instances.

The scheduler changes deployment as inter-instance bandwidth changes.

Request-Time Breakdown in Table 5

THUNDERSERVE / METHODOLOGY

LLaMA-30B, 4×A40 + 4× 3090Ti. High / low are the 40 Gbps and 5 Gbps deployments on the previous slide.

Configuration	Prefill	KV comm	Decode	Throughput
Baseline	884 ms	0 ms	1689 ms	1610 tok/s
ThunderServe, 40 Gbps	698 ms	133 ms	1126 ms	3292 tok/s
ThunderServe, 5 Gbps	964 ms	41 ms	1846 ms	2196 tok/s

The 5 Gbps row has less KV time because the deployment stops sending KV across instances, not because the slow link is faster.

Do not invert one request's prefill + decode time and call that system throughput.

Source: Appendix H, Table 5.

Throughput Depends on Prefill-to-Decode Ratio

- LLaMA-13B with 8, 12 and 16 A5000 GPUs; two GPUs per replica.
- Ratio means number of prefill replicas to decode replicas.
- Coding requests have longer prompts and shorter outputs, favoring more prefill replicas.
- Conversation requests have shorter prompts and longer outputs, favoring more decode replicas.

The optimal phase ratio changes with workload and cluster size.

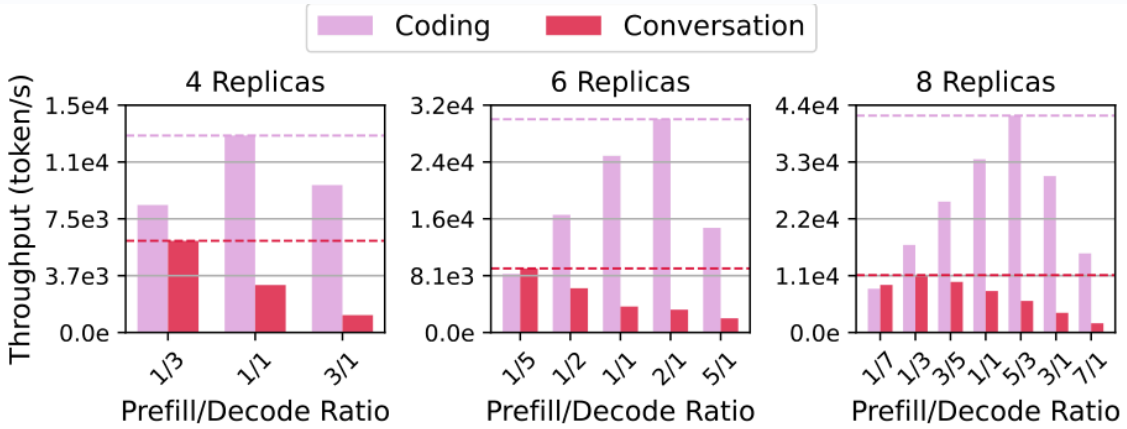


Figure 6. Throughput (token/s) by prefill-to-decode ratio. Impact of phase designation and orchestration on overall system throughput. We experiment with LLaMA-13B on both coding and conversation workloads across clusters with 8, 12, and 16 A5000 GPUs, respectively, with two GPUs serving one replica. The ratio represents the prefill-to-decode ratio (i.e., the ratio of # prefill replicas to # decode replicas). We have also provided SLO Attainment results in Figure 14 of Appendix D.

Fig. 6: Throughput vs. phase ratio

SLO Attainment Also Depends on Phase Ratio

THUNDERSERVE / METHODOLOGY

- LLaMA-13B on 16 A5000 GPUs; two GPUs per replica.
- Coding workload: 5:3 is the reported optimal prefill-to-decode ratio.
- Conversation workload: 3:5 is the reported best ratio.
- More decode replicas help with longer-running decoding.

Lightweight re-scheduling adjusts phase designation for workload shifts.

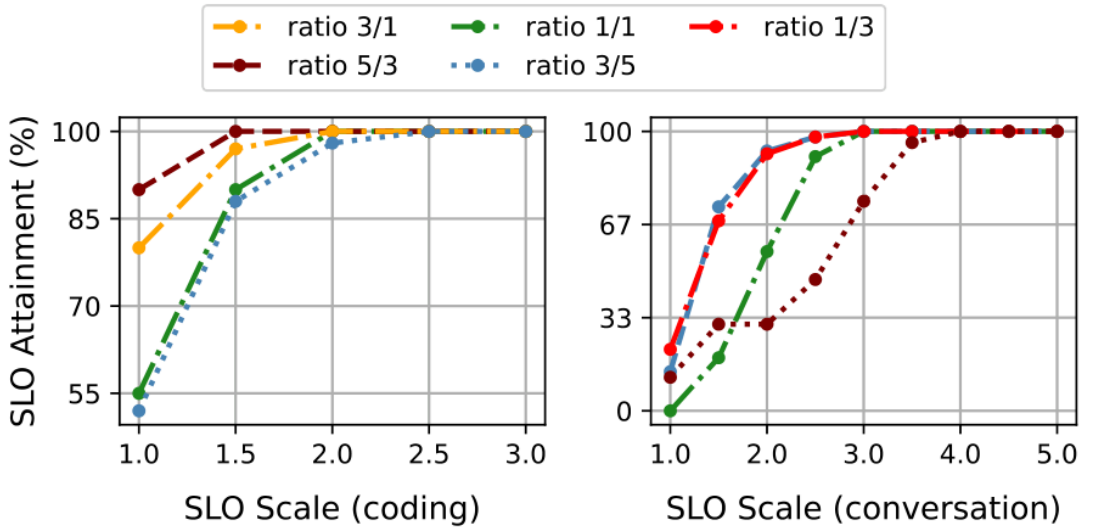


Figure 14. Impact of phase designation and orchestration on overall system SLO attainment. We experiment with LLaMA-13B on both coding and conversation workloads across 16 A5000 GPUs, with two GPUs serving one replica.

Fig. 14: Phase designation and orchestration (Appendix D)

Lightweight Re-scheduling in Real Time

THUNDERSERVE / METHODOLOGY

WHY RE-SCHEDULE?

Workload variability

Workload characteristics and GPU availability change in online cloud services.

PARALLELISM

Skip new deduction

Keep surviving group construction and existing TP / PP configurations.

TABU SEARCH

Only flipping phase designation

Do not use splitting, merging or moving groups during lightweight re-scheduling.

ORCHESTRATION

Solve routing again

Optimize orchestration with updated phase designation; no model-parameter reloading.

Keeping weights in place saves reload time, but restricts the plans that can be explored.

Source: Section 3.4. Quantitative overhead appears in Table 4.

Runtime Support for Phase Splitting

THUNDERSERVE / METHODOLOGY

COMMUNICATION SETUP

Pre-created NCCL group pool

Establish potentially needed communication groups in advance to reduce group-creation overhead.

QUEUE LOCATION

KV buffers on prefill replicas

Completed prompt KV caches wait in prefill GPU memory.

TRANSFER

Asynchronous KV communication

Use asynchronous communication to overlap KV transfers with computation.

DECODE SIDE

Retrieve KV after a decoding round

Decode replicas pull queued KV caches; the deployment plan guides request assignment.

The runtime implements the plan while managing KV queues and communication overhead.

Source: Section 4 and Appendix E; FlashAttention and PagedAttention are also integrated.

Quantize KV Cache for Communication Only

THUNDERSERVE / METHODOLOGY

- Quantize and pack 16-bit KV values into 4-bit elements for transfer.
- Decode replica unpacks and dequantizes received KV cache.
- Both prefill and decode computations use 16-bit KV cache.
- LLaMA-7B, two A5000s, 40 Gbps: communication share falls from 16–30% to 4–9%.

Dequantization restores the 16-bit representation, not the information lost in quantization.

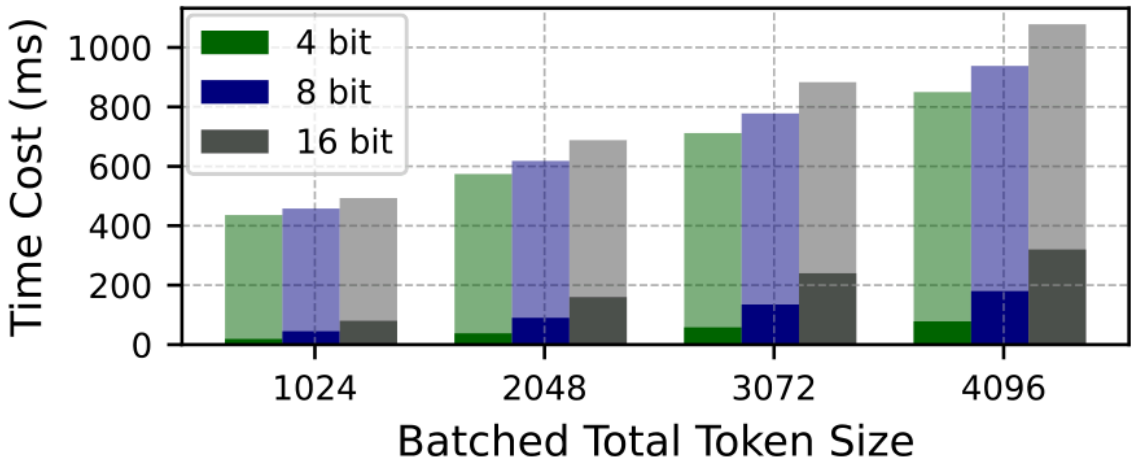


Figure 18. Impact of KV cache communication compression. (Non-transparent: time cost of KV cache communication. Transparent: end-to-end processing time.)

Fig. 18: Impact of KV cache communication compression

KV Compression: Speed and Model Quality

THUNDERSERVE / EXPERIMENTAL EVIDENCE

LLaMA-30B, Appendix H setup (Table 8)

Precision	Prefill	KV transfer	Decode	Throughput
16-bit	684 ms	584 ms	1108 ms	2450 tok/s
4-bit	698 ms	133 ms	1126 ms	3292 tok/s ($1.34\times$)

- Table 2: largest observed task-score drop is GSM8K / LLaMA-13B, $22.34 \rightarrow 21.29$ (1.05 percentage points; about 4.7% relative).
- Tables 6–7: PPL changes by less than 1%; ROUGE against 16-bit outputs ranges from 0.911 to 0.975 in these tests.

The tested quality changes are small in absolute terms, but compression is lossy.

Source: Table 2 and Appendix I, Tables 6–8. Relative changes calculated from table values.

Task Scores under 4-bit KV Transfer

THUNDERSERVE / EXPERIMENTAL EVIDENCE

Table 2. CoQA, TruthfulQA and GSM8K. Scores are task accuracy, not latency.

Task	Precision	LLaMA-7B	LLaMA-13B
CoQA	16-bit / 4-bit	63.95 / 64.58	66.35 / 66.54
TruthfulQA	16-bit / 4-bit	30.64 / 30.13	29.68 / 29.34
GSM8K	16-bit / 4-bit	13.23 / 12.54	22.34 / 21.29

CoQA rises slightly. The largest absolute drop is GSM8K / 13B, $22.34 \rightarrow 21.29$ (1.05 points, about 4.7% relative). GSM8K / 7B drops about 5.2% relative.

``Below 2%'' in the text matches these absolute point drops, not every relative drop.

Source: Table 2. Relative changes are calculated from the cells.

PPL and ROUGE under 4-bit KV Transfer

THUNDERSERVE / EXPERIMENTAL EVIDENCE

Table 6. PPL, 16-bit / 4-bit. Lower is better.

Dataset	LLaMA-7B	LLaMA-30B
WikiText2	3.53 / 3.55	2.73 / 2.75
PTB	7.46 / 7.42	6.49 / 6.55
CBT	7.66 / 7.70	6.31 / 6.30

Table 7. ROUGE of 4-bit outputs vs. 16-bit outputs.

Dataset	Metric	7B	30B
WikiText2	R-1 / R-2 / R-L	.962/.941/.955	.942/.928/.941
PTB	R-1 / R-2 / R-L	.975/.950/.971	.928/.911/.928
CBT	R-1 / R-2 / R-L	.925/.912/.925	.946/.931/.937

PPL changes stay within about 1%. ROUGE spans 0.911 (PTB, 30B, ROUGE-2) to 0.975 (PTB, 7B, ROUGE-1).

ROUGE here measures agreement with 16-bit outputs, not factual correctness.

Source: Appendix I, Tables 6–7.

Experimental Setup and Baselines

THUNDERSERVE / EXPERIMENTAL RESULTS

HETEROGENEOUS CLOUD

32 GPUs / $13.542 per hour

Two 4-A6000, two 4-A5000, one 8-A40 and two 4-3090Ti instances (Vast.ai).

MODEL / WORKLOAD

LLaMA-30B / Azure Conversation

Coding and conversation workloads; Poisson arrivals with exponential inter-arrival times.

HOMOGENEOUS IN-HOUSE

8 A100-80GB / $14.024 per hour

A comparable cloud-rental price budget for vLLM and DistServe baselines.

BASELINES / METRICS

HexGen, vLLM, DistServe

TTFT, TPOT and E2E SLO attainment; system throughput.

HexGen uses the same cloud hardware; vLLM / DistServe use a similar-price A100 server.

Source: Section 5.1 and Table 1. Prices are the paper's reported rental prices, not current quotes.

Reading SLO Attainment and Latency Deadlines

THUNDERSERVE / EXPERIMENTAL RESULTS

- **TTFT:** request arrival to first token; includes waiting and prompt processing.
- **TPOT:** average time per subsequent output token during decoding.
- **E2E latency:** request arrival to completion, including waiting and inter-phase transfer.
- **SLO attainment:** fraction of requests meeting the specified deadline; the paper evaluates TTFT, TPOT and E2E separately.
- **SLO scale:** deadline divided by the A100 execution-latency reference used in the evaluation.

At the same request rate and target attainment, a curve farther left needs a shorter deadline.

Example: a $2\times$ deadline improvement means $d_{\text{baseline}}/d_{\text{ThunderServe}} = 2$ at the same attainment. It is not a claim about mean latency.

Source: Sections 2 and 5.1.

Cloud SLO Attainment: ThunderServe vs. HexGen

THUNDERSERVE / EXPERIMENTAL RESULTS

- ThunderServe improves TTFT, TPOT and E2E SLO attainment over HexGen.
- Compare minimum deadlines at the same arrival rate and target attainment.
- Coding: E2E deadline ratio (HexGen / ThunderServe) up to $1.8\times$; average $1.4\times$.
- Conversation: E2E deadline ratio up to $1.4\times$; average $1.3\times$.

Phase splitting reduces interference between prefill and decode.

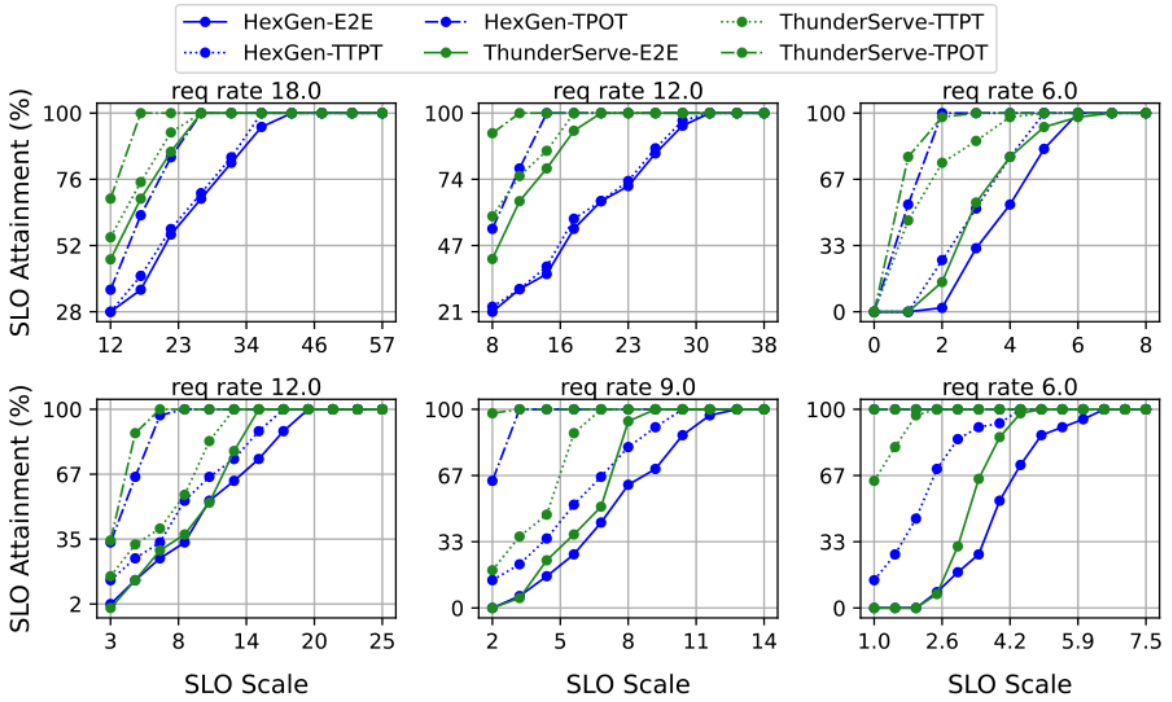


Figure 7. SLO attainment results on coding (top row) and conversation (bottom row) workloads.

Fig. 7: Coding (top) and conversation (bottom) SLO

Cost Efficiency: Cloud vs. In-House

THUNDERSERVE / EXPERIMENTAL RESULTS

- ThunderServe on cloud is compared with DistServe and vLLM on in-house A100 GPUs.
- The total cloud price budget is close to the 8-A100 baseline budget.
- E2E deadline ratio (baseline / ThunderServe): up to $2.5\times$; average $1.8\times$ (Section 5.2).
- 12 cloud phase replicas versus 4 DistServe phase replicas; this is a system-plus-hardware comparison.

More replicas and heterogeneity-aware scheduling improve cost efficiency.

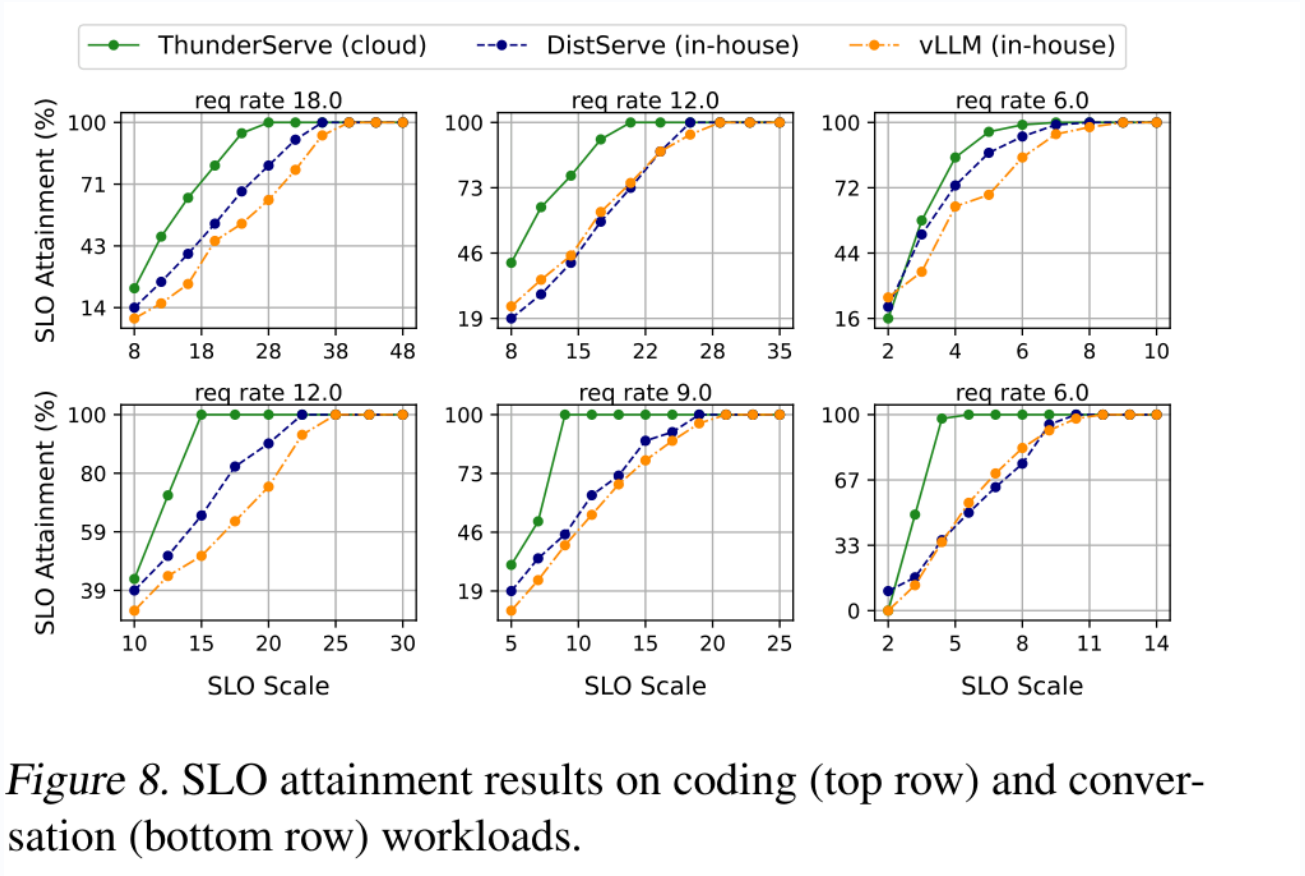


Figure 8. SLO attainment results on coding (top row) and conversation (bottom row) workloads.

Fig. 8: SLO attainment under similar price budgets

End-to-End System Throughput

THUNDERSERVE / EXPERIMENTAL RESULTS

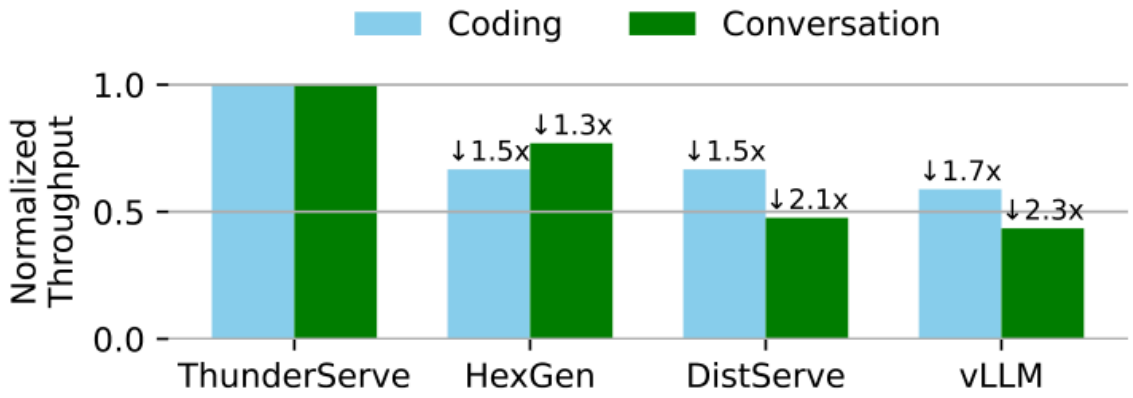


Figure 9. Throughput scaled by ThunderServe.

Fig. 9: Throughput scaled by ThunderServe

- Against HexGen: $1.5\times$ (coding) and $1.3\times$ (conversation) throughput.
- Against DistServe: $1.5\times$ (coding) and $2.1\times$ (conversation).
- Fig. 9 also reports $1.7\times$ / $2.3\times$ versus vLLM (coding / conversation).
- Fig. 9 normalizes ThunderServe to 1.0; its vLLM $2.3\times$ exceeds the abstract's stated $2.1\times$ maximum.

Use baseline-specific ratios; the abstract and Fig. 9 do not state a consistent overall maximum.

Deployment Plans Found on the 32-GPU Cluster

THUNDERSERVE / EXPERIMENTAL RESULTS

	Coding	Conversation
Prefill replicas	8	4
Decode replicas	4	8
Total phase replicas	12	12

- Coding: 8 A40 GPUs form four prefill replicas, each with TP=2, PP=1.
- Conversation: 6 A40 GPUs form three prefill replicas; 2 A40 GPUs form one decode replica with TP=1, PP=2.
- A mixed replica uses 2 A5000 + 2 3090 Ti with TP=2, PP=2: TP stays within a GPU type, while PP connects stages.

Replica count, phase and parallel configuration all respond to the workload.

Source: Table 3 and Appendix F. These are full deployment plans, not a phase-flip-only transition.

Full Deployment Plans from Table 3

THUNDERSERVE / EXPERIMENTAL RESULTS

Workload	GPUs	Strategy	Replicas
Coding	$8\times$A40	TP=2, PP=1	4 prefill
	$4\times$A5000	TP=4, PP=1	1 prefill
	$4\times$A6000	TP=2, PP=1	2 prefill
	$2\times$A5000+$2\times$ 3090Ti	TP=2, PP=2	1 prefill
	$4\times$ 3090Ti	TP=2, PP=2	1 decode
	$4\times$A6000	TP=1, PP=2	2 decode
	$2\times$A5000+$2\times$ 3090Ti	TP=2, PP=2	1 decode
Conversation	$6\times$A40	TP=2, PP=1	3 prefill
	$2\times$A5000+$2\times$ 3090Ti	TP=2, PP=2	1 prefill
	$4\times$ 3090Ti	TP=2, PP=2	1 decode
	$2\times$A40	TP=1, PP=2	1 decode
	$4\times$A5000	TP=2, PP=2	1 decode
	$8\times$A6000	TP=1, PP=2	4 decode
	$2\times$A5000+$2\times$ 3090Ti	TP=2, PP=2	1 decode

Coding is 8 prefill + 4 decode; conversation is 4 prefill + 8 decode. Both use 12 replicas.

Source: Table 3 and Appendix F. The previous slide highlighted only the A40 and one mixed row.

Tabu Search: Scheduling Convergence

THUNDERSERVE / EXPERIMENTAL RESULTS

- Scheduling from scratch is evaluated using 16, 24 and 32 GPUs.
- Approximate convergence times: 21, 36 and 54 seconds.
- The search runs once before initial model deployment.
- Scheduling time is small relative to the hourly scale of online services.

Upper-level scheduling scales to the heterogeneous 32-GPU testbed.

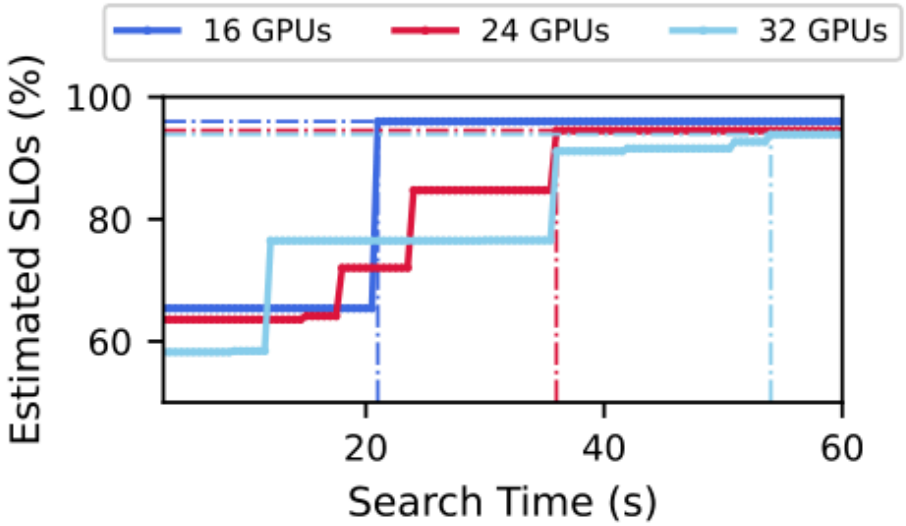


Figure 10. Convergence curves of scheduling from scratch for different cluster sizes.

Fig. 10: Convergence under different GPU cluster sizes

Lightweight Re-scheduling after GPU Failures

THUNDERSERVE / EXPERIMENTAL RESULTS

- Test: one 4-A6000 instance goes offline, removing two decode replicas.
- Lightweight re-scheduling yields SLO attainment comparable to full re-scheduling.
- Table 4: full search $54(\pm 5)$ s plus reload $103(\pm 10)$ s, totaling $157(\pm 13)$ s.
- Lightweight search $13(\pm 2)$ s and reload 0 s, so the whole step is $13(\pm 2)$ s.

Measured re-scheduling overhead is about $12\times$ lower; only weight reloading takes zero time.

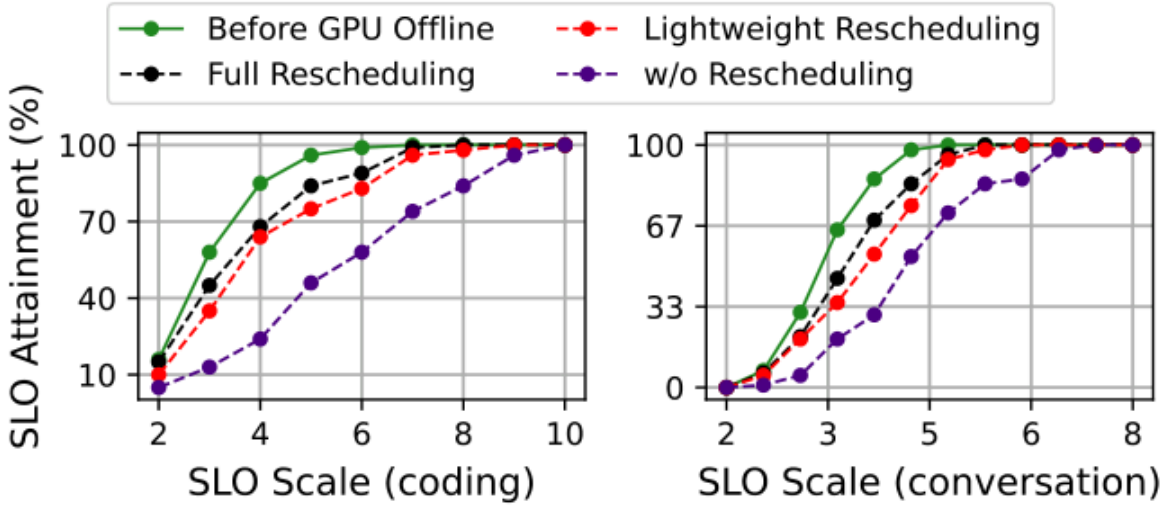


Figure 11. SLO attainments of before (indicated by the solid line) and after (indicated by the dotted lines) 4 out of 32 GPUs offline.

Fig. 11: SLO with 4 of 32 GPUs offline

Ablation: KV Cache Compression and Orchestration

THUNDERSERVE / EXPERIMENTAL RESULTS

- Without KV cache compression, paper reports roughly $1.3\times$ per-request overhead.
- Reduced communication efficiency weakens phase splitting benefits.
- Replacing the orchestration method with random dispatching causes further performance degradation.
- This removal is cumulative: random dispatch is tested with compression already disabled.

The ablation supports both components; it does not isolate orchestration with compression enabled.

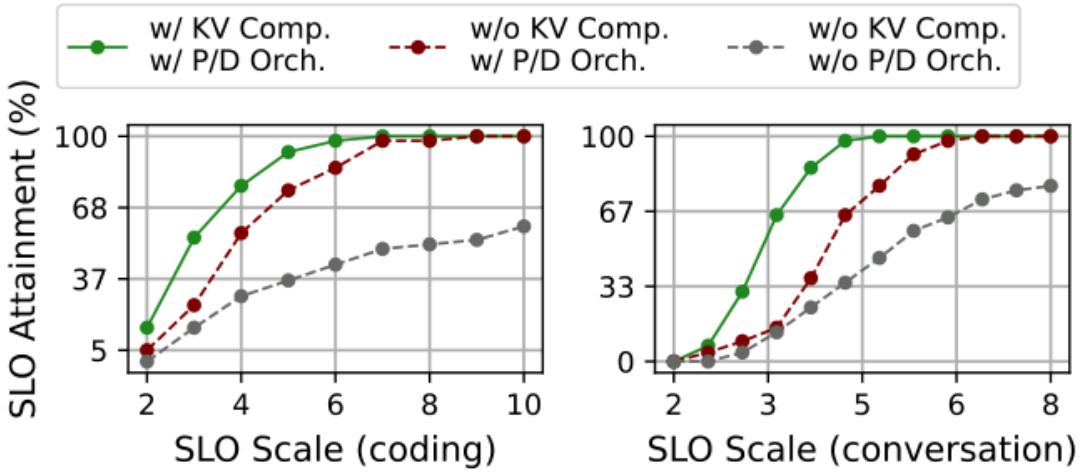


Figure 12. Impact of KV cache compression and prefill and decode orchestration on the SLO attainments.

Fig. 12: Component ablations and SLO attainment

Simulator and Alpha-Beta Model Accuracy

THUNDERSERVE / EXPERIMENTAL RESULTS

- Appendix J uses LLaMA-30B micro-benchmarks.
- Vary SLO scales and batched token sizes.
- Left: simulated attainment differs from measurements by at most 2 percentage points in the plotted cases.
- Right: alpha-beta latency differs by at most 2 ms across the four plotted token sizes.

These micro-benchmarks support the estimates in the tested settings; they are not a general error bound.

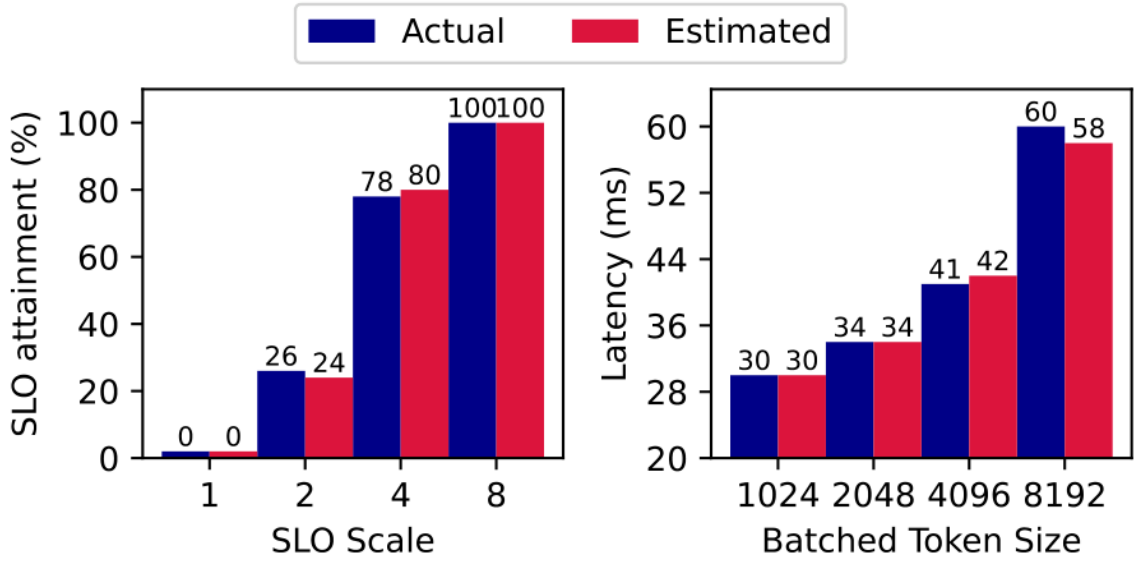


Figure 19. Comparison of benchmarked and estimated performance metrics for simulator (left) and alpha-beta model (right).

Fig. 19: Benchmarked vs. estimated performance

Discussion: Scope and Open Questions

THUNDERSERVE / DISCUSSION

- **Search quality:** tabu search and constrained TP / PP enumeration do not guarantee a globally optimal deployment.
- **Fast adaptation:** keeping groups and parallelism fixed reduces disruption, but cannot explore every plan after a workload shift.
- **Evidence scope:** main results use LLaMA-30B, two workload families and one 32-GPU cloud setup; broader models and network conditions need validation.
- **Reproducibility:** routing constraints and load dependence of pairwise scores need more detail than the printed formulation provides.

Research question: when should a service pay for full re-planning instead of only changing phase roles?

Presenter analysis based on Sections 3 and 5; the final question is not an evaluated paper result.

ThunderServe: Contributions and Evaluation Findings

THUNDERSERVE / CONCLUSION

SCHEDULING

Two-level hierarchical optimization

Plan GPU groups, phase roles, TP / PP and request routing.

COMMUNICATION

KV cache compression

Quantize for inter-phase transfer; dequantize before decode computations.

RUNTIME

Lightweight re-scheduling

Adjust phases and orchestration without reloading model parameters.

SYSTEM

Heterogeneous cloud serving

Combine TP / PP with phase splitting across heterogeneous cloud GPUs.

Selected results: vs. DistServe: $1.5\times$ / $2.1\times$ throughput (coding / conversation); cloud-vs-in-house E2E deadline ratio up to $2.5\times$.

ThunderServe integrates phase splitting with heterogeneous cloud GPUs and network-aware scheduling.

Source: Sections 5.2 and 6; Fig. 9. Deadline ratios compare matched SLO attainment.

Questions & Discussion

Phase splitting / Scheduling / KV transfer

ThunderServe: heterogeneous cloud serving